Konfiguration eines Programms

Carsten Gips (HSBI)

TL;DR
Zu Konfiguration von Programmen kann man beim Aufruf Kommandozeilenparameter mitgeben. Diese sind in der über den Parameter`String[] args` in der `main(String[] args)`-Methode zugreifbar.
Es gibt oft eine Kurzversion ("-x") und/oder eine Langversion ("–breite"). Zusätzlich können Parameter noch ein Argument haben ("-x 12" oder "–breite=12"). Parameter können optional oder verpflichtend sein.
Um dies nicht manuell auswerten zu müssen, kann man beispielsweise die Bibliothkek Apache Commons CLI benutzen.
Ein anderer Weg zur Konfiguration sind Konfigurationsdateien, die man entsprechend einliest. Hier findet man häufig das "Ini-Format", also zeilenweise "Key=Value"-Paare. Diese kann man mit der Klasse `java.util.Properties` einlesen, bearbeiten und speichern (auch als XML).
Videos

- VL Konfiguration
- Demo Setzen von Kommandozeilenparametern (IDE)
- Demo Einbinden von Libs in IDE
- Demo Apache Commons CLI
- Demo Properties

---

1 Wie kann man Programme konfigurieren?

1. Parameter beim Start mitgeben: Kommandozeilenparameter (CLI)
2. Konfigurationsdatei einlesen und auswerten

2 Varianten von Kommandozeilenparameter

- Fixe Reihenfolge

  ```
  java MyApp 10 20 hello debug
  ```

- Benannte Parameter I

  ```
  java MyApp -x 10 -y 20 -answer hello -d
  ```

- Benannte Parameter II

  ```
  java MyApp --breite=10 --hoehe=20 --answer=hello --debug
  ```

Häufig Mischung von Kurz- und Langformen

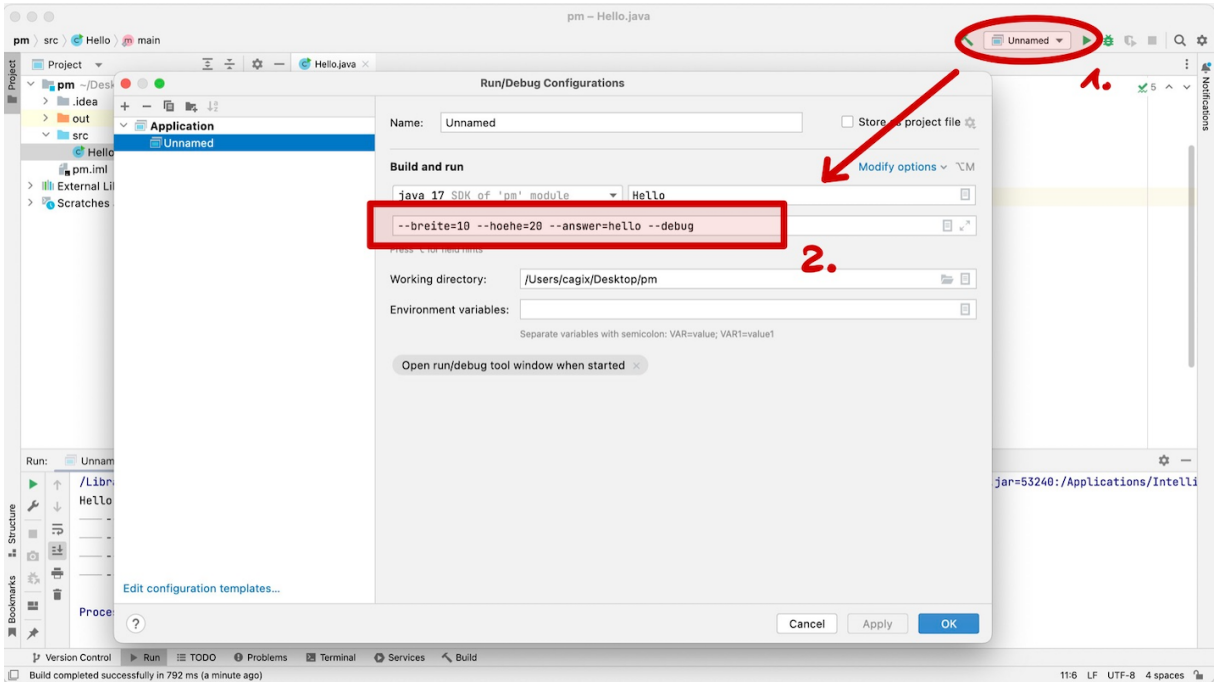
Häufig hat man eine Kurzform der Optionen, also etwa "-x". Dabei ist der Name der Option in der Regel ein Zeichen lang. Es gibt aber auch Abweichungen von dieser Konvention, denken Sie beispielsweise an `java -version`.

In der Langform nutzt man dann einen aussagekräftigen Namen und stellt zwei Bindestriche voran, also beispielsweise “–breite” (als Alternative für “-x”).

Wenn Optionen Parameter haben, schreibt man in der Kurzform üblicherweise “-x 10” (trennt also den Parameter mit einem Leerzeichen von der Option) und in der Langform “–breite=10” (also mit einem “=” zwischen Option und Parameter). Das sind ebenfalls Konventionen, d.h. man kann prinzipiell auch in der Kurzform das “=” nutzen, also “-x=10”, oder in der Langform mit einem Leerzeichen trennen, also “–breite 10”.

Demo IDE und CLI (https://youtu.be/a3XUfDbD9uo)

Hinweis IntelliJ: “`Edit Configurations`” => Kommandozeilenparameter unter “`Build and run`” im entsprechenden Feld eintragen



3 Auswertung Kommandozeilenparameter

- Kommandozeilenparameter werden als String-Array an `main()`-Methode übergeben:

```
public static void main(String[] args) { }
public static void main(String... argv) { }
```

=> Müssen “händisch” ausgewertet werden

Anmerkung: Nur Parameter! Nicht Programmname als erster Eintrag wie in C ...

4 Beispiel Auswertung Kommandozeilenparameter

```
public static void main(String[] args) {
    int x = 100;
    String answer = "";
    boolean debug = false;

    // Parameter: -x=10 -answer=hello -debug
    // => args = ["-x=10", "-answer=hello", "-debug"]
    for (String param : args) {
        if (param.startsWith("-x")) { x = Integer.parseInt(param.substring(3));
            }
        if (param.startsWith("-a")) { answer = param.substring(8); }
        if (param.startsWith("-d")) { debug = true; }
    }
}
```

4.1 Kritik an manueller Auswertung Kommandozeilenparameter

- Umständlich und unübersichtlich
- Große `if-else`-Gebilde in `main()`
- Kurz- und Langform müssen getrennt realisiert werden
- Optionale Parameter müssen anders geprüft werden als Pflichtparameter
- Überlappende Parameternamen schwer aufzufinden
- Prüfung auf korrekten Typ nötig bei Parametern mit Werten
- Hilfe bei Fehlern muss separat realisiert und gepflegt werden

5 Apache Commons: CLI

Rad nicht neu erfinden!

- Apache Commons bietet die CLI-Bibliothek zum Umgang mit Kommandozeilenparametern an: commons.apache.org/cli

Annäherung an fremde API:

- Lesen der verfügbaren Doku (PDF, HTML)
- Lesen der verfügbaren Javadoc
- Herunterladen der Bibliothek
- Einbinden ins Projekt

6 Exkurs: Einbinden fremder Bibliotheken/APIs

6.1 Eclipse

- Lib von commons.apache.org herunterladen und auspacken
- Neuen Unterordner im Projekt anlegen: `libs/`
- Bibliothek (`.jar`-Files) hinein kopieren
- Projektexplorer, Kontextmenü auf `.jar`-File: "`Add as Library`"
- Alternativ Menü-Leiste: "`Project > Properties > Java Build Path > Libraries > Add JARs`"

6.2 IntelliJ

- Variante 1:
  - Lib von commons.apache.org herunterladen und auspacken
  - Neuen Unterordner im Projekt anlegen: `libs/`
  - Bibliothek (`.jar`-Files) hinein kopieren
  - Variante 1 (a):Projektexplorer, Kontextmenü auf `.jar`-File: "`Build Path > Add to Build Path`"
  - Variante 1 (b): Projekteigenschaften, Eintrag "Libraries", "+", "New Project Library", "Java" und Jar-File auswählen
- Variante 2:
  - Projekteigenschaften, Eintrag "Libraries", "+", "New Project Library", "From Maven" und "commons-cli:commons-cli:1.5.0" als Suchstring eingeben und die Suche abschließen

6.3 Gradle oder Ant oder Maven

- Lib auf Maven Central suchen: "commons-cli:commons-cli" als Suchstring eingeben
- Passenden Dependency-Eintrag in das Build-Skript kopieren

6.4 Kommandozeilenaufruf

- Class-Path bei Aufruf setzen:
  - Unix: `java -cp .:<jarfile>:<jarfile> <mainclass>`
  - Windows: `java -cp .;<jarfile>;<jarfile> <mainclass>`

  Achtung: Unter Unix (Linux, MacOS) wird ein Doppelpunkt zum Trennen der Jar-Files eingesetzt, unter Windows ein Semikolon!

Beispiel: `java -classpath .:/home/user/wuppy.jar MyApp`

Vorgriff auf Build-Skripte (spätere VL): Im hier gezeigten Vorgehen werden die Abhängigkeiten manuell aufgelöst, d.h. die Jar-Files werden manuell heruntergeladen (oder selbst kompiliert) und dem Projekt hinzugefügt.

Alle später besprochenen Build-Skripte (Ant, Gradle) beherrschen die automatische Auflösung von Abhängigkeiten. Dazu muss im Skript die Abhängigkeit auf geeignete Weise beschrieben werden und wird dann beim Kompilieren des Programms automatisch von spezialisierten Servern in der im Skript definierten Version heruntergeladen. Dies funktioniert auch bei rekursiven Abhängigkeiten …

7 Überblick Umgang mit Apache Commons CLI

Paket: `org.apache.commons.cli`

1. Definition der Optionen
   - Je Option eine Instanz der Klasse `Option`
   - Alle Optionen in Container `Options` sammeln
2. Parsen der Eingaben mit `DefaultParser`
3. Abfragen der Ergebnisse: `CommandLine`
4. Formatierte Hilfe ausgeben: `HelpFormatter`

Die Funktionsweise der einzelnen Klassen wird in der Demo kurz angerissen. Schauen Sie bitte zusätzlich in die Dokumentation.

Demo: Einbinden von Libs, cli.Args (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/java-jvm/src/cli/Args.java)

8 Laden und Speichern von Konfigurationsdaten

```
#ola - ein Kommentar
hoehe=2
breite=9
gewicht=12
```

- Konfigurationsdaten sind i.d.R. Schlüssel-Wert-Paare (`String`/`String`)

  => `java.util.Properties`

  Tatsächlich verbirgt sich ein `Hashtable` dahinter:

  ```
  public class Properties extends Hashtable<Object,Object>;
  ```

9 Laden und Speichern von Konfigurationsdaten (cnt.)

- Properties anlegen und modifizieren

```
Properties props = new Properties();
props.setProperty("breite", "9");
props.setProperty("breite", "99");
String value = props.getProperty("breite");
```

- Properties speichern: `Properties#store` und `Properties#storeToXML`

```
public void store(Writer writer, String comments)
public void store(OutputStream out, String comments)
public void storeToXML(OutputStream os, String comment, String encoding)
```

- Properties laden: `Properties#load` und `Properties#loadFromXML`

```
public void load(Reader reader)
public void load(InputStream inStream)
public void loadFromXML(InputStream in)
```

Demo: cli.Props, Hinweis auf “Apache Commons Configuration” (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/java-jvm/src/cli/Props.java)

`java.util.Properties` sind eine einfache und im JDK bereits eingebaute Möglichkeit, mit Konfigurationsdateien zu hantieren. Deutlich umfangreichere Möglichkeiten bieten aber externe Bibliotheken, beispielsweise “Apache Commons Configuration” (commons.apache.org/configuration).

10 Wrap-Up

- Kommandozeilenparameter als `String[]` in `main()`-Methode
- Manuelle Auswertung komplex => *Apache Commons CLI*
- Schlüssel-Wert-Paare mit `java.util.Properties` aus/in Dateien laden/speichern

11 Zum Nachlesen

- Oracle Corporation (2022)

Lernziele

- k3: Auswertung von Kommandozeilenparametern in einem Programm
- k3: Apache Commons CLI zur Verarbeitung von Kommandozeilenparametern
- k3: Laden von Konfigurationsdaten mit java.util.Properties

Quizzes

- Quiz Konfiguration (ILIAS)

Quellen

Oracle Corporation. 2022. „The Java Tutorials“. 2022. https://docs.oracle.com/javase/tutorial/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 71232c0 (tooling: shift headings (use h1 as top-level headings), 2025-04-29)